

The Cross-Model Handoff Developer Method

A practical workflow for transferring software-development work between AI coding agents, models, and vendors without sacrificing continuity, verification, or engineering quality.

Core idea: treat an AI model switch like an engineering shift change. The outgoing agent leaves a verified checkpoint and explicit handoff; the incoming agent validates that state before continuing.

Status: Experimental developer method. Adapt it to your repository, risk profile, tooling, and team. The method is intentionally vendor-neutral.

1. Why this method exists

Modern AI coding agents can implement substantial amounts of software quickly, but model context is temporary and vendor usage limits, context limits, outages, cost constraints, or model strengths can force a developer to switch agents mid-project. A careless switch can create duplicated work, incompatible architecture, overwritten changes, and subtle regressions.

The Cross-Model Handoff Method makes the repository - not the conversation - the durable source of project truth. The outgoing model externalizes its temporary knowledge while it still has it. The incoming model verifies that knowledge against Git, source code, builds, and tests before writing new code.

2. The operating principle

Speed is the optimization. Engineering quality is the constraint.

The objective is not to maximize generated lines of code. It is to minimize **time-to-reliable-software**. Fast implementation is valuable only when the resulting state can be reproduced, tested, understood, and safely inherited by another developer or agent.

3. The handoff cycle

Phase	Outgoing agent / developer action
1. Stop	Stop starting new functionality before the model, credit, or context limit is exhausted.
2. Stabilize	Finish or safely suspend the current unit of work. Avoid leaving unexplained half-edits.
3. Verify	Build, run relevant tests, record failures, and inspect Git status.
4. Checkpoint	Create a coherent commit or clearly document uncommitted work.
5. Externalize	Update AGENT_HANDOFF.md with current state, decisions, risks, and exact next actions.
6. Transfer	Give the next model the repository instructions plus the handoff.
7. Re-verify	Incoming model inspects Git and independently reruns verification before changing code.
8. Continue	Only after reconciliation does the incoming model implement the next task.

4. Recommended repository artifacts

AGENTS.md (or equivalent) is the relatively stable engineering contract: architecture, code style, supported platforms, build commands, test expectations, forbidden shortcuts, dependency policy, and definition of done.

AGENT_HANDOFF.md is temporary operational state. It should be rewritten as the project moves. It is not a replacement for Git history, issues, tests, or architecture documentation.

```
# AGENT_HANDOFF.md

Current branch / commit:
Current objective:
Completed:
```

```
Partially completed:
Exact remaining TODOs:
Files / modules touched:
Architecture decisions and assumptions:
Do NOT change without review:
Known bugs / technical debt:
Build command:
Test command:
Latest build result:
Latest test result:
Uncommitted changes:
Recommended first action:
Notes for the next agent:
```

5. Outgoing-agent prompt

Stop implementing new functionality and prepare a cross-model handoff.

1. Bring the repository to the safest practical checkpoint.
2. Inspect git status and summarize all modifications.
3. Run the relevant build and test commands.
4. Do not hide failures; record exact failures and suspected causes.
5. Update AGENT_HANDOFF.md with completed work, partial work, remaining TODOs, files touched, architectural decisions, known problems, verification results, and the recommended first action for the next agent.
6. Do not begin the next feature after completing the handoff.

6. Incoming-agent prompt

Before modifying code:

1. Read AGENTS.md, project documentation, and AGENT_HANDOFF.md.
2. Inspect git status, recent commits, and the files referenced by the handoff.
3. Independently run the documented build/tests.
4. Treat source code, Git history, and reproducible test results as more authoritative than the previous model's prose.
5. If the handoff conflicts with repository reality, document the discrepancy and reconcile it before continuing.
6. Continue from the first verified incomplete task while preserving established architecture unless a change is explicitly justified.

7. Example: quota-based model switch

A developer is using Model A to implement a C++ spreadsheet engine. The model has approximately ten minutes of usable quota remaining. Instead of starting the next UI feature, the developer asks Model A to stop, run CMake builds and tests, record the current commit, and produce the handoff. Model B is then opened with the repository and told to validate the handoff. Model B reruns the tests and continues only after confirming the checkpoint.

This is especially useful when a developer maintains subscriptions or credit pools with multiple vendors and wants to use them as interchangeable development capacity without allowing architectural context to disappear at each switch.

8. Example: models with different strengths

The method does not need to be triggered by a quota. One model can perform implementation, another can perform an independent review, and a third can focus on testing or documentation. The handoff becomes a boundary between responsibilities rather than merely a recovery mechanism.

```
Model A: implement feature -> verify -> handoff
Model B: review implementation -> identify issues -> handoff
Model C: implement/fix tests -> verify -> handoff
Human: inspect results -> approve/merge -> choose next objective
```

9. Parallel work: use isolation

Do not use a handoff document as permission for several agents to edit the same working tree simultaneously. For parallel work, isolate agents with separate Git branches or worktrees and assign non-overlapping responsibilities where possible. Merge only after CI/review. Sequential handoffs are safer when tasks share architecture or files.

10. Failure modes the method is designed to reduce

Context reconstruction: the next model wastes time rediscovering what happened. **Architecture drift:** a new model unknowingly introduces a competing pattern. **False completion:** prose says a feature is finished while tests disagree. **Half-written work:** an agent expires midway through a refactor. **Duplicate implementation:** the next agent recreates functionality that already exists. **Cross-coding:** multiple agents alter shared interfaces at the same time.

11. Rules worth enforcing

Keep handoffs factual and short enough to be read. Record exact commands instead of saying 'tests work.' Never manufacture a green checkpoint by disabling tests. Prefer small coherent commits. Make failures visible. Keep permanent architecture decisions outside the temporary handoff when they deserve durable documentation. Require the incoming model to verify rather than blindly trust.

12. Experiments to try

Independent verification: have Model B receive only the repository and handoff, then measure how often it reproduces Model A's claimed results. **Reviewer rotation:** alternate which model implements and which

reviews to see whether different models catch different classes of defects. **Handoff compression:** compare short structured handoffs with verbose session summaries and measure time-to-resume and error rate.

Task specialization: create roles such as Implementer, Test Engineer, Performance Reviewer, Security Reviewer, or Documentation Agent. **Context-loss simulation:** deliberately start a fresh session using only repository artifacts and see whether work can continue without conversational history. **Metrics:** track build failures after handoff, regressions, duplicated work, tokens/credits consumed, time-to-resume, and human intervention required.

13. A stronger definition of done

A task is not complete because an agent says it is complete. Completion should be demonstrated through the evidence appropriate to the project: successful builds, automated tests, static analysis, sanitizers, benchmarks, integration checks, reproducible packaging, or manual acceptance criteria.

The handoff is a map. The repository and reproducible verification are the territory.

14. When not to switch models

Avoid switching in the middle of a tightly coupled refactor, database migration, unresolved merge conflict, or architectural redesign unless the outgoing agent can first produce a safe checkpoint. If the developer is tired enough that the next prompt may be poorly specified, stopping at a verified checkpoint can be faster overall than creating cleanup work.

15. Minimal version

For small projects, the entire method can be reduced to five actions: **stop, test, commit, handoff, verify**. The process should scale with project risk rather than becoming bureaucracy for its own sake.

16. Closing principle

Cross-model development works best when no model is treated as the sole holder of project knowledge. Models are replaceable execution and reasoning resources. Durable intent belongs in repository documentation; durable state belongs in version control; confidence belongs in reproducible verification; final responsibility remains with the developer or team operating the system.

This method is intentionally open-ended. Fork it, shorten it, automate it, measure it, and publish improvements that make multi-agent development safer or faster.